

Assignment Report

Student: Denisenko Sofiia 狄苏菲

Student id: 1820249051

Course: Computer Organization and Architecture

Assignment №2

1.

Given C code as

```
while (save[i] == k)
    i += 1;
```

where i in x22, k in x24, address of save in x25, data type is double word.
What is the compiled RISC-V code?

(Tips.

- ① Double word contains 64 bits, and the main memory is byte-addressing.
- ② Use bne/beq to implement the loop.)

```
Loop: slli x10, x22, 3      // Shift Left Logic  $i \cdot 2^3(8)$ 
      add x10, x10, x25     // adding the base address of save
      ld x(9), 0(x10)      // loading the value into x9
      bne x9, x24, Exit    // branch is not equal -> Exit
      addi x22, x22, 1     // if equal, then doing  $i+=1$ 
      beq x0, x0, Loop     // unconditional, Loop
```

2.

Suppose the machine code for each instruction has 16 bits, and the address code for one operand includes 6 bits. There are 3 kinds instructions: zero operand, one operand and two operands' instructions.

- (1) If the number of zero operand instructions is L, the number of two operands instructions is K, then what's the maximum number of one operand instructions?

Each instruction takes 16 bits. Each operand address uses 6 bits.

- Zero operand instruction requires 16 bits.
- One operand instruction requires 16 bits and 6 bits for the operand address. This means $16 - 6 = 10$ bits are left for the opcode and other fields.
- Two operand instruction requires 16 bits and 12 bits for two operand addresses (6 bits each). This leaves 4 bits for the opcode and other fields.

To maximize the number of one-operand instructions (**M**), we need to minimize the number of bits used by zero(**L**) and two-operand(**K**) instructions to free up as many bits as possible.

Let's assume we want to maximize the number of zero-operand instructions, so we use the entire 16 bits for the opcode. This allows us to have $2^{16} = 65,536$ unique zero-operand instructions.

For the two-operand instructions it's $2^4 = 16$.

$$(L * 16) + (M * 16) + (K * 16) = 16 * (L + M + K) // \text{Total bits used}$$

$$16 * (L + M + K) - (L * 16) - (K * 16) = 16 * M // \text{Available bits for one-operand instructions}$$

Since we have 10 bits for the opcode of one-operand instructions, the maximum M is $2^{10} = 1024$, but we need to consider the total bits used for all instructions. We must have $16 * (L + M + K) \leq 2^{16}$ (total bits).

$$16 * (65536 + M + 16) \leq 2^{16}$$

$$M \leq 1022.$$

Answer: the maximum number of one-operand instructions is **1022**.

(2) What are the maximum number of instructions for each of these 3 kinds instructions?

1: Maximizing zero operand instructions (L):

$$L = 2^{16 - 1 - 1} = 2^{14} = 16,384$$

$$M = 1$$

$$K = 1$$

2: Maximizing one operand instructions (M):

$$L = 1$$

$$M = 2^{16 - 1 - 1 - 6} = 2^8 = 256$$

$$K = 1$$

3: Maximizing two operand instructions (K):

$$L = 1$$

$$M = 1$$

$$K = 2^{16 - 1 - 1 - 12} = 2^2 = 4$$

3.

Suppose the machine code for each instruction has 12 bits, and the address code for one operand includes 3 bits. Please give a scheme such that there are 4 instructions with three addresses, 8 instructions with two addresses, and 180 instructions with zero address.

Zero address instructions:

Since we have 12 bits per instruction, we can use 7 bits for the opcode ($2^7 = 128$, which is less than 180, but we can use the remaining 5 bits to encode the remaining 52 instructions). Therefore, we can encode 128 instructions in the first 7 bits.

Two address instructions:

These instructions will use the Opcode field, Operand 1, and Operand 2. Since we have 7 bits for opcode (already used for zero address instructions), we have 5 bits left to represent the 8 different two-address instructions. This is sufficient, as $2^5 = 32$ greater than 8. To represent them, we use 2 bits for opcode (to represent 4 unique two-address instruction types). The remaining 3 bits of opcode (used for zero-address instructions) are used to differentiate the 8 different instructions (with 0 as the least significant bit, and 7 as the most significant bit).

Three address instructions:

These instructions will use all fields (Opcode, Operand 1, Operand 2, and Operand 3). We have already allocated 7 bits for the Opcode field. Since there are 4 different three-address instructions, we need 2 bits to encode them. The remaining 3 bits of the Opcode are used to distinguish them. To represent them, we use 1 bit for opcode (to represent 2 unique three-address instruction types). The remaining 6 bits of opcode (used for zero-address instructions) are used to differentiate the 4 different instructions (with 0 as the least significant bit, and 3 as the most significant bit).

Bit allocation:

<i>Instruction type</i>	<i>Opcode (bits)</i>	<i>Operand 1 (bits)</i>	<i>Operand 2 (bits)</i>	<i>Operand 3 (bits)</i>	<i>Total bits</i>
<i>Zero address</i>	7	-	-	-	7
<i>Two address</i>	2	3	3	-	8
<i>Three address</i>	1	3	3	3	10

4.

Suppose the word length 16 bits, the main memory contains 64K words, the instruction is as word length and contain only one address. There are 64 instructions in total, please state:

(1) If using the direct addressing, how many main memory units can be accessed?

Direct addressing:

Since the main memory has 2^{16} words, we need 16 bits to address each word.

Each instruction has one address field, and since the word length is 16 bits, the address field can also hold 16 bits.

With direct addressing, the address field directly points to the memory location.

Answer: with **direct addressing**, we can access **all 64K words** of the main memory.

(2) If using both direct and indirect addressing, with one bit as the identification for direct or indirect mode, what is the range of main memory that can be accessed? What is the range of main memory that can be accessed by the direct addressing?

Direct and indirect addressing:

We use 15 bits for direct addressing, as we need to use 1 bit to specify direct/indirect addressing mode.

In indirect addressing, the address field points to a memory location that contains the actual address of the data. Since the address field is 15 bits, it can address $2^{15} = 32K$ words. However, since the actual address is stored in another memory location, the indirect addressing mode can access all 64K words of the main memory.

Answer: with **both direct and indirect addressing**, we can access all **64K words of the main memory**. The **direct addressing range** is limited to **32K words** because we use 15 bits for direct addressing.

5.

Assume the 12-bits instruction has the format

11	10	9	8	7	6	5	4	3	2	1	0
@	OP	I ₁	I ₂	Z/C	A						

Bit11=1: non-direct addressing

Bit8=1: index addressing with register I₁

Bit7=1: index addressing with register I₂

Bit6: Z/C =0 for page addressing with page number as 0; Z/C =1 for page addressing with page number as the page number of PC

Assume the main memory has 2^{12} storage unit, it is divide into 2^6 pages, and each page has 2^6 words.

The contents of these registers are (Q denotes for the octal):

(PC)=0340Q

(I₁)=1111Q

(I₂)=0256Q

Please calculate the executive address for the following instructions.

(1) 1046Q

(2) 2433Q

(3) 3215Q

(4) 1111Q

1. Instruction 1046Q

Binary: 001 000 100 110

Interpretation:

Bit 11: 0

Bit 8: 0, Bit 7: 0 (No index addressing)

Bit 6: 0 (Page 0)

Executive address: 100110 = **38 (Decimal)**

2. Instruction 2433Q

Binary: 010 100 011 011

Interpretation:

Bit 11: 0

Bit 8: 1 (Index addressing with I₁)

Bit 6: 0 (Page 0)

Address:

Base Address: 011011 = 16 + 8 + 3 = 27 (Decimal)

Index: I₁ = 1111Q = 511 (Decimal)

Executive Address: 27 + 585 = **612 (Decimal)**

3. Instruction 3215Q

Binary: 011 010 001 101

Interpretation:

Bit 11: 0

Bit 7: 1 (Index Addressing with I2)

Bit 6: 0 (Page 0)

Address:

Base address: $001101 = 8 + 4 + 1 = 13$ (Decimal)

Index: $I2 = 0256Q = 174$ (Decimal)

Executive address: $13 + 174 = 187$ (Decimal)

4. Instruction 1111Q

Binary: 001 001 001 001

Interpretation:

Bit 11: 0

Bit 8: 0, Bit 7: 0 (No index addressing)

Bit 6: 1 (Page PC)

Address:

Base address: $001001 = 8 + 1 = 9$ (Decimal)

Page number from PC: $PC = 0340Q = 224$ (Decimal). Dividing by words per page (2^6) to get the page number: $224/64 = 3$

Address of page: $3 * 2^6 = 3 * 64 = 192$

Executive address: $9 + 192 = 201$ (Decimal)